

```

import json, os, smtplib, sys
from datetime import datetime
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText

GMAIL_ADDRESS = os.environ.get("GMAIL_ADDRESS", "")
GMAIL_APP_PASS = os.environ.get("GMAIL_APP_PASS", "")
SEND_TO = os.environ.get("SEND_TO", "")
SITE_URL = os.environ.get("SITE_URL", "https://your-site.netlify.app")
DATA_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "adp-da
POS_COLORS = {"QB": "#f97316", "RB": "#22d3ee", "WR": "#a78bfa", "TE": "#34d3
POSITIONS = ["QB", "RB", "WR", "TE"]

def load_data():
    if not os.path.exists(DATA_FILE):
        print("ERROR: adp-data.json not found.")
        sys.exit(1)
    with open(DATA_FILE) as f:
        return json.load(f)

def get_movers(players, n=10, pos=None, alltime=False):
    movers = []
    for p in players:
        if pos and p["pos"] != pos:
            continue
        h = p.get("history", [])
        if len(h) < 2:
            continue
        if alltime:
            delta = round(h[-1]["adp"] - h[0]["adp"], 1)
            days = len(h)
        else:
            delta = round(h[-1]["adp"] - h[-2]["adp"], 1)
            days = 0
        if delta == 0:
            continue
        movers.append({
            "name": p["name"],
            "team": p.get("team", ""),
            "pos": p["pos"],
            "adp": h[-1]["adp"],
            "delta": delta,

```

```

        "days": days,
    })
    risers = sorted([m for m in movers if m["delta"] < 0], key=lambda x: x["delta"])
    fallers = sorted([m for m in movers if m["delta"] > 0], key=lambda x: -x["delta"])
    return risers, fallers

def badge(pos):
    c = POS_COLORS.get(pos, "#999")
    return (
        ' <span style="padding:2px 7px;border-radius:4px;font-size:10px;'
        'font-weight:700;background:' + c + '22;color:' + c + ';font-family:monospace;'
        + pos + '</span>'
    )

def row(p, up, show_days=False):
    arrow = "up" if up else "dn"
    dc = "#4ade80" if up else "#f87171"
    ar = "&#9650;" if up else "&#9660;"
    days_str = ""
    if show_days and p.get("days"):
        days_str = '<div style="color:#6b6560;font-size:10px">' + str(p["days"])
    return (
        '<tr style="border-bottom:1px solid #2a2520">'
        '<td style="padding:9px 8px;width:44px">' + badge(p["pos"]) + '</td>'
        '<td style="padding:9px 8px">'
        '<div style="font-weight:600;color:#f0ede8;font-size:13px">' + p["name"]
        '<div style="color:#6b6560;font-size:11px">' + p["team"] + days_str + '</div>'
        '</td>'
        '<td style="padding:9px 8px;text-align:right;font-family:monospace;font-size:12px">'
        + str(round(p["adp"], 1)) +
        '</td>'
        '<td style="padding:9px 8px;text-align:right;font-family:monospace;font-size:12px">'
        + ar + ' ' + str(abs(p["delta"])) +
        '</td>'
        '</tr>'
    )

def table(rows_html):
    return (
        '<table style="width:100%;border-collapse:collapse;background:#1a1714;'
        'border-radius:12px;overflow:hidden;border:1px solid #2a2520">'
        + rows_html + '</table>'
    )

```

```

def label(text, color):
    return (
        '<div style="font-size:11px;font-weight:700;color:' + color + ';"'
        'letter-spacing:0.1em;text-transform:uppercase;margin-bottom:8px;margin-t'
        + text + '</div>'
    )

def divider(text):
    return (
        '<div style="margin:24px 0 8px;text-align:center;font-size:10px;'
        'color:#6b6560;font-weight:700;letter-spacing:0.12em;text-transform:upper'
        'border-top:1px solid #2a2520;padding-top:16px">' + text + '</div>'
    )

def build_html(players, updated):
    today = datetime.now().strftime("%A, %B %d").replace(" 0", " ")

    dr, df = get_movers(players, 10)
    ar, af = get_movers(players, 10, alltime=True)
    days_tracked = max((len(p.get("history", [])) for p in players), default=1)

    body = ""

    if dr or df:
        body += divider("Today's Top Movers")
        if dr:
            body += label("&#9650; Top " + str(len(dr)) + " Risers", "#4ade80")
            body += table("").join(row(p, True) for p in dr)
        if df:
            body += label("&#9660; Top " + str(len(df)) + " Fallers", "#f87171")
            body += table("").join(row(p, False) for p in df)

    body += divider("By Position – Top 5")
    for pos in POSITIONS:
        pr, pf = get_movers(players, 5, pos=pos)
        if not pr and not pf:
            continue
        pc = POS_COLORS[pos]
        body += '<div style="font-size:12px;font-weight:700;color:' + pc + ';"marg
        if pr:
            body += label("&#9650; Risers", "#4ade80")
            body += table("").join(row(p, True) for p in pr)
        if pf:
            body += label("&#9660; Fallers", "#f87171")

```

```

        body += table("".join(row(p, False) for p in pf))

if ar or af:
    body += divider("All-Time Since Day 1 (" + str(days_tracked) + " days)")
    if ar:
        body += label("&#9650; Biggest Risers", "#4ade80")
        body += table("".join(row(p, True, True) for p in ar))
    if af:
        body += label("&#9660; Biggest Fallers", "#f87171")
        body += table("".join(row(p, False, True) for p in af))

if not body:
    return None

return (
    '<!DOCTYPE html><html><head><meta charset="UTF-8"/></head>'
    '<body style="margin:0;padding:0;background:#0f0d0b;font-family:Helvetica'
    '<div style="max-width:520px;margin:0 auto;padding:24px 16px">'
    '<div style="margin-bottom:8px">'
    '<div style="font-size:22px;font-weight:800;color:#f0ede8;letter-spacing:'
    'ADP<span style="color:#f97316">&#176;</span>TRACKER</div>'
    '<div style="font-size:11px;color:#6b6560;letter-spacing:0.08em;margin-to'
    'UNDERDOG FANTASY &middot; NFL BEST BALL &middot; ' + today.upper() + '</'
    '</div>'
    + body +
    '<div style="text-align:center;margin:28px 0 20px">'
    '<a href="' + SITE_URL + '" style="display:inline-block;background:#f9731'
    'color:#0f0d0b;font-weight:700;font-size:14px;padding:14px 32px;'
    'border-radius:10px;text-decoration:none">View Full ADP Tracker &#8594;</'
    '</div>'
    '<div style="font-size:11px;color:#4a4540;text-align:center;'
    'border-top:1px solid #2a2520;padding-top:16px">'
    'Data via FantasyPros / Underdog Fantasy &middot; Updated ' + updated + '
    '</div></body></html>'
)

def send_email(html, dr, df):
    if not GMAIL_ADDRESS or not GMAIL_APP_PASS or not SEND_TO:
        print("ERROR: Missing email env vars.")
        sys.exit(1)
    today = datetime.now().strftime("%b %d").replace(" 0", " ")
    top_r = dr[0]["name"] if dr else "---"
    top_f = df[0]["name"] if df else "---"
    subject = "ADP Daily Movers " + today + " -- Up: " + top_r + " | Dn: " + top_f
    msg = MIME multipart("alternative")
    msg["Subject"] = subject

```

```

msg["From"] = GMAIL_ADDRESS
msg["To"] = SEND_TO
msg.attach(MIMEText(html, "html"))
print("Sending to " + SEND_TO + "...")
with smtplib.SMTP_SSL("smtp.gmail.com", 465) as s:
    s.login(GMAIL_ADDRESS, GMAIL_APP_PASS)
    s.sendmail(GMAIL_ADDRESS, SEND_TO, msg.as_string())
print("Email sent!")

def main():
    data = load_data()
    players = data.get("players", [])
    updated = data.get("updated", "")
    print("Loaded " + str(len(players)) + " players.")
    dr, df = get_movers(players, 10)
    print(str(len(dr)) + " risers, " + str(len(df)) + " fallers")
    html = build_html(players, updated)
    if not html:
        print("No movers yet -- needs 2+ days of data.")
        sys.exit(0)
    send_email(html, dr, df)

if __name__ == "__main__":
    main()

```